

PHYS-4040: Elementary Particle Physics

---

# Project

---

Ashiful★Bhuiyan

Instructor: Prof. Nikita Blinov

Submission Date: April 10, 2025



Simulations of Two Body Decays

# Simulating Two Body Decays

When an energetic beam of protons collides with a target material, it produces lots of particles, many of which are pions. We want to simulate the decay of  $\pi^0 \rightarrow \gamma\gamma$ .

## 1 Theory

We need to consider the relativistic kinematics and some quantum field theory.

Our neutral pion is a short-lived meson composed of a quantum superposition of up-antiup ( $u\bar{u}$ ) and down-antidown ( $d\bar{d}$ ) quarks.

$$\pi^0 = \frac{1}{\sqrt{2}}(u\bar{u} - d\bar{d})$$

It decays via the **electromagnetic interaction** into two photons:

$$\pi^0 \rightarrow \gamma\gamma$$

### 1.1 Conservation Laws of $\pi^0$ at rest

- ▷ Energy needs to be conserved:  $E_{initial} = E_{final}$ . The energy conservation of a pion at rest is then:

$$\begin{aligned} E &= pc \\ E_{\pi^0} &= m_{\pi^0}c^2 \\ E_{\pi^0} = E_\gamma + E_\gamma &= \frac{m_{\pi^0}c^2}{2} + \frac{m_{\pi^0}c^2}{2} \\ p_1 = p_2 &= \frac{m_{\pi^0}c}{2} \end{aligned}$$

- ▷ momentum needs to be conserved: In the pion's rest frame, the total momentum of the decay products, must be zero. So the two photons are emitted in opposite directions with equal magnitudes of momentum:

$$\vec{p}_{\pi^0} = 0 = \vec{p}_1 + \vec{p}_2 \implies \vec{p}_1 = -\vec{p}_2$$

- ▷ angular momentum needs to be conserved: pion has spin-0, so two photons must have angular momentum 0. Since photons have spin-1, their helicity is  $\pm 1$ , so the spins must be opposite to cancel each other out.
- ▷ parity needs to be conserved: Pion has negative parity, so the two photon system must match that.
- ▷ charge needs to be conserved: Photons are neutral, so charge is conserved.

## 1.2 Kinematics of Two Body Decay

In the rest frame of our pion, it has mass  $m_{\pi^0} = 135$  MeV and no initial momentum. After decaying into the two photons, the photons travel at the speed of light,  $c$  just in opposite directions of one another.

In general for a particle at rest  $p^\mu = (mc, 0, 0, 0)$  with energy  $E = mc^2$  decaying to two particles with momenta  $p_1$  and  $p_2$ :

$$\begin{aligned} E &= E_1 + E_2 \\ mc^2 &= E_1 + E_2 \end{aligned}$$

We can solve for the momenta using the energy-momentum relation:  $E^2 = p^2c^2 + m^2c^4$  for each particle.

$$\begin{aligned} E_1^2 &= p_1^2c^2 + m_1^2c^4 \\ E_2^2 &= p_2^2c^2 + m_2^2c^4 \end{aligned}$$

Squaring both sides and using energy conservation:

$$m^2c^4 = (E_1 + E_2)^2 = E_1^2 + E_2^2 + 2E_1E_2$$

Substituting  $E_1^2$  and  $E_2^2$ :

$$m^2c^4 = p_1^2c^2 + m_1^2c^4 + p_2^2c^2 + m_2^2c^4 + 2E_1E_2$$

And the following:

$$m^2c^4 - m_1^2c^4 - m_2^2c^4 = p_1^2c^2 + p_2^2c^2 + 2E_1E_2$$

$$p_1 = p_2 = p$$

$$m^2c^4 - m_1^2c^4 - m_2^2c^4 = 2p^2c^2 + 2E_1E_2$$

Now solve for  $E_1E_2$ :

$$E_1E_2 = \frac{(m^2 + m_1^2 - m_2^2)(m^2 + m_2^2 - m_1^2)}{4m^2}c^4$$

Lastly we have the following for the magnitude of the momentum of the daughter particles of the decay:

$$p = \frac{\sqrt{(m^2 - (m_1 + m_2)^2)(m^2 - (m_1 - m_2)^2)}}{2m}c$$

When the pion is not at rest, i.e has momentum  $p_{\pi^0}$ , we can first analyze the decay in the rest frame of the pion like we did just have, then apply a Lorentz boost to the decay products to the lab frame.

Suppose pion has an initial 4-momenta:  $p^\mu = (\gamma mc, \gamma vm, 0, 0)$  where  $\gamma = \frac{1}{\sqrt{1-v^2}}$ . Then, the decay products must be Lorentz-boosted to the lab frame.

$$\begin{aligned} E' &= \gamma(E - vp_x) \\ p'_x &= \gamma(p_x + vE) \end{aligned}$$

## 1.3 Decay Rates

The decay  $\pi^0 \rightarrow \gamma\gamma$  is given by:

$$\Gamma_{\pi \rightarrow \gamma\gamma} = \frac{(2\pi)^4}{2E_\pi} \int |M_{\pi \rightarrow \gamma\gamma}|^2 \delta(E_\pi - E_{\gamma 1} - E_{\gamma 2}) \delta^3(\mathbf{p}_\pi - \mathbf{p}_{\gamma 1} - \mathbf{p}_{\gamma 2}) \frac{d^3\mathbf{p}_{\gamma 1}}{(2\pi)^3 2E_{\gamma 1}} \frac{d^3\mathbf{p}_{\gamma 2}}{(2\pi)^3 2E_{\gamma 2}}$$

For this we need the matrix element, which we can get by analyzing the interaction of the specific coupling of the pion to photons. After we have that, we will need to perform the integration to get the total decay rate. This means we will need to know our detector's acceptance and other experimental constraints.

The neutral pion is composed of up and down quarks:  $\pi^0 = \frac{1}{\sqrt{2}}(u\bar{u} - d\bar{d})$ . We get an effective Lagrangian for the pion-photon interaction:

$$\mathcal{L}_{\pi^0\gamma\gamma} = \frac{\alpha}{8\pi f_\pi} \pi^0 F_{\mu\nu} \bar{F}^{\mu\nu}$$

From this alongside with the Feynman rules, we can calculate the decay width:

$$\Gamma(\pi^0 \rightarrow \gamma\gamma) = \frac{\alpha^2 m_{\pi^0}}{64\pi^3 f_\pi^2} \approx 7.8 \text{ eV}$$

The branching ratio is:

$$\text{BR}(\pi^0 \rightarrow \gamma\gamma) = \frac{\Gamma(\pi^0 \rightarrow \gamma\gamma)}{\Gamma_{\text{total}}} \approx 98.8\%$$

In our simulation we are only considering this decay mode, and ignoring the other decays such as the Dalitz decay ( $\pi^0 \rightarrow e^+e^-\gamma$ ) for example.

## 2 Lorentz product, invariant masses, and Lorentz Transformation

### 2.1 Lorentz product

The Lorentz product/Minkowski inner product of two four-vectors  $A^\mu$  and  $B^\mu$  is:

$$A \cdot B = A^\mu B_\mu = A^0 B^0 - \vec{A} \cdot \vec{B} = A^0 B^0 - A^1 B^1 - A^2 B^2 - A^3 B^3$$

```
1 def lorentz_product(A, B):
2     """Computes the Lorentz product of two four-vectors."""
3     return A[0] * B[0] - np.dot(A[1:], B[1:])
```

To test the Lorentz product, we can use some known properties and simple cases:

1. Self-product gives squared invariant mass:

$$P \cdot P = m^2$$

```
1 #testing the functions
2 P = np.array([5, 3, 4, 0]) # E = 5, p = (3,4,0)
3 expected_mass_squared = 5**2 - (3**2 + 4**2) # Should be 0 (massless
4   case)
5 computed_mass_squared = lorentz_product(P, P)
6 print(computed_mass_squared, expected_mass_squared,
7       expected_mass_squared-computed_mass_squared)
8 assert np.isclose(computed_mass_squared, expected_mass_squared), "
9     Invariant mass test failed!"
```

Output:

```
1 0 0 0
```

2. We can also check if two orthogonal four-vectors  $A^\mu$  and  $B^\mu$  have a Lorentz product of zero:

$$A \cdot B = 0$$

```
1 A = np.array([2, 1, 1, 0])
2 B = np.array([1, 1, -1, 0]) # Manually chosen so that their Lorentz
  product is 0
3 assert np.isclose(lorentz_product(A, B), 0), "Orthogonality test failed
  !"
```

Output:

```
1 0 0 0
```

and lastly, can check that the Lorentz product remains unchanged after a Lorentz boost:

```
1 P = np.array([10, 3, 4, 0])
2 velocity = np.array([0.6, 0, 0]) # Boost along x-direction
3 P_boosted = lorentz_transform(P, velocity)
4 print(lorentz_product(P, P), lorentz_product(P_boosted, P_boosted))
5 assert np.isclose(lorentz_product(P, P), lorentz_product(P_boosted,
  P_boosted)), "Lorentz invariance test failed!"
```

Output:

```
1 75 75.0
```

So, the functions are all behaving like we expect them to. We can then say, this product is invariant under Lorentz transformations meaning it has the same value in all inertial frames of references.

## 2.2 Invariant mass

The invariant mass of a particle with four-momentum vector  $P^\mu = E, \vec{P}$  is defined as:

$$\begin{aligned} m^2 c^2 &= P \cdot P = P^\mu P_\mu = P^0 P^0 - \vec{P} \cdot \vec{P} = E^2 - \vec{P}^2 \\ m^2 c^2 &= E^2 - \vec{p} \cdot \vec{p} \end{aligned}$$

and with  $c = 1$ :  $m^2 = E^2 - \vec{p} \cdot \vec{p}$

For a pair of particles with four-momentum vectors  $P_1^\mu$  and  $P_2^\mu$ , with system mass  $M$ :

$$\begin{aligned} M^2 &= (P_1 + P_2)^2 = (P_1 + P_2) \cdot (P_1 + P_2) \\ M^2 &= P_1 \cdot P_1 + P_2 \cdot P_2 + 2P_1 \cdot P_2 \\ M^2 &= m_1^2 + m_2^2 + 2P_1 \cdot P_2 \end{aligned}$$

where  $m_1$  and  $m_2$  are the invariant masses of the two particles.

The code for the invariant mass and system mass:

```

1 def invariant_mass(P):
2     """Computes the invariant mass of a four-momentum vector P."""
3     return np.sqrt(P[0]**2 - np.dot(P[1:], P[1:]))
4
5 def system_mass(P1, P2):
6     """Computes the invariant mass of a system of two particles."""
7     P_total = P1 + P2
8     return np.sqrt(lorentz_product(P_total, P_total))

```

```

1 def test_invariant_mass():
2     P = np.array([5, 3, 4, 0])
3     expected_mass = np.sqrt(5**2 - (3**2 + 4**2))
4     assert np.isclose(invariant_mass(P), expected_mass), "Invariant
    mass test failed!"

```

Output:

```

1 Invariant mass test passed!

```

## 2.3 Lorentz Transformation

The Lorentz transformation matrix for a boost in an arbitrary direction  $\mathbf{v} = (v_x, v_y, v_z)$  is given by:

$$\Lambda^\mu_\nu = \begin{pmatrix} \gamma & -\gamma\beta n_x & -\gamma\beta n_y & -\gamma\beta n_z \\ -\gamma\beta n_x & 1 + (\gamma - 1)n_x^2 & (\gamma - 1)n_x n_y & (\gamma - 1)n_x n_z \\ -\gamma\beta n_y & (\gamma - 1)n_y n_x & 1 + (\gamma - 1)n_y^2 & (\gamma - 1)n_y n_z \\ -\gamma\beta n_z & (\gamma - 1)n_z n_x & (\gamma - 1)n_z n_y & 1 + (\gamma - 1)n_z^2 \end{pmatrix}$$

where  $\beta = \frac{v}{c}$ ,  $\gamma = \frac{1}{\sqrt{1-\beta^2}}$ , and  $\mathbf{n} = (n_x, n_y, n_z) = \frac{\mathbf{v}}{v}$ .

The code for this is:

```

1 def lorentz_transform(P, v):
2     """Applies a Lorentz boost to a four-vector P in the direction of
3     velocity v."""
4     v = np.array(v)
5     v_mag = np.linalg.norm(v)
6     if v_mag == 0:
7         return P # No boost needed
8
9     beta = v_mag / 1 # Assume c=1
10    gamma = 1 / np.sqrt(1 - beta**2)
11    n = v / v_mag # Normalize direction
12
13    # Lorentz transformation matrix
14    Lambda = np.array([
15        [gamma, -gamma * beta * n[0], -gamma * beta * n[1], -gamma *
16         beta * n[2]],
17        [-gamma * beta * n[0], 1 + (gamma - 1) * n[0]**2, (gamma - 1) *
18         n[0] * n[1], (gamma - 1) * n[0] * n[2]],
19        [-gamma * beta * n[1], (gamma - 1) * n[1] * n[0], 1 + (gamma -
20         1) * n[1]**2, (gamma - 1) * n[1] * n[2]],
21        [-gamma * beta * n[2], (gamma - 1) * n[2] * n[0], (gamma - 1) *
22         n[2] * n[1], 1 + (gamma - 1) * n[2]**2]
23    ])

```

```

19 |
20 |     return np.dot(Lambda, P)

```

We have already verified the Lorentz transform using our Lorentz product. So we can just refer to that for the check of this function.

### 3 Two Body Decay of a particle of mass $M$ at rest

When a particle of mass  $M$  decays into two particles with mass  $m_1$  and  $m_2$ , the total energy and momentum must be conserved. In the rest frame of the parent particle, the total energy before the decay is  $M$  and the total momentum before decay is 0. As we discussed before, the decay products will move in opposite directions with equal and opposite momenta.

Using the energy-momentum relation:  $E^2 - p^2 = m^2$ , we can find the energy and momentum of the decay products.

$$E_1 = \frac{M^2 + m_1^2 - m_2^2}{2M}, \quad E_2 = \frac{M^2 + m_2^2 - m_1^2}{2M}$$

The magnitude of the momenta are then:

$$p = \sqrt{E_1^2 - m_1^2} = \sqrt{E_2^2 - m_2^2}$$

To ensure we get an isotropic decay (equal probability in all directions), we can sample the polar angle  $\theta$  from a uniform distribution in  $\cos \theta = 2\text{rand}() - 1$  (this gives us values from  $[-1,1]$ ) and the azimuthal angle  $\phi$  uniformly from 0 to  $2\pi$ .

Then we convert to cartesian coordinates:

$$p_x = p \sin \theta \cos \phi, \quad p_y = p \sin \theta \sin \phi, \quad p_z = p \cos \theta$$

These are the momenta of the decay product of the first particle and the second particle is simply the negative of the first.

```

1 def two_body_decay(M, m1, m2):
2     """Generates two random four-momenta for a two-body decay in the
3     rest frame of the decaying particle."""
4     E1 = (M**2 + m1**2 - m2**2) / (2 * M)
5     E2 = (M**2 + m2**2 - m1**2) / (2 * M)
6     p_mag = np.sqrt(E1**2 - m1**2) # Magnitude of the momentum
7
8     # Generate a random isotropic direction
9     theta = np.arccos(2 * np.random.rand() - 1) # Uniform in cos(theta)
10
11     phi = 2 * np.pi * np.random.rand() # Uniform in phi
12
13     p1 = np.array([E1, p_mag * np.sin(theta) * np.cos(phi), p_mag * np.
14     sin(theta) * np.sin(phi), p_mag * np.cos(theta)])
15     p2 = np.array([E2, -p1[1], -p1[2], -p1[3]]) # Opposite direction
16
17     return p1, p2

```

We did not need to know the matrix element to compute the kinematics of the two-body decay because we are dealing with conservation laws and relativistic constraints. This allows to get the momentum distribution.

If we were to apply dynamics with the Lagrangian, we would need to find the matrix element. If we wanted to generate more correct physics-based events, we would need to consider the actual probabilistic distribution of the decays rather than assuming our events are uniform.

## 4 Mass $M$ moving with velocity $v$

After calculating the momentum of the daughter particles in the rest frame, we need to boost to the lab frame to each daughters four-momentum.

$$P'^{\mu} = \Lambda_n^{\mu} u P^{\nu}$$

```

1 def two_body_decay(M, m1, m2, v):
2     """Generates two random four-momenta for a two-body decay in the
3     rest frame of the decaying particle,
4     then boosts them to the lab frame where the decaying particle moves
5     with velocity v."""
6     E1 = (M**2 + m1**2 - m2**2) / (2 * M)
7     E2 = (M**2 + m2**2 - m1**2) / (2 * M)
8     p_mag = np.sqrt(E1**2 - m1**2) # Magnitude of the momentum
9
10    # Generate a random isotropic direction
11    theta = np.arccos(2 * np.random.rand() - 1) # Uniform in cos(theta)
12    phi = 2 * np.pi * np.random.rand() # Uniform in phi
13
14    p1 = np.array([E1, p_mag * np.sin(theta) * np.cos(phi), p_mag * np.
15    sin(theta) * np.sin(phi), p_mag * np.cos(theta)])
16    p2 = np.array([E2, -p1[1], -p1[2], -p1[3]]) # Opposite direction
17
18    # Boost to the lab frame
19    p1_lab = lorentz_transform(p1, v)
20    p2_lab = lorentz_transform(p2, v)
21
22    return p1_lab, p2_lab

```

## Some Validation of the Decays

We take some random 4-momenta decay the pion and check the conservation of energy and momentum of the products.

$$E_{\pi^0} = E_1 + E_2, \vec{p}_{\pi^0} = \vec{p}_1 + \vec{p}_2$$

To see how well these hold, compute the energy and momentum differences and plot their distributions:

$$E_{\pi^0} - (E_1 + E_2), |\vec{p}_{\pi^0} - (\vec{p}_1 + \vec{p}_2)|$$

```

1 # Conservation checks
2 df["energy_diff"] = df["E"] - (df["E1"] + df["E2"])
3 df["momentum_diff"] = np.sqrt(
4     (df["px"] - (df["px1"] + df["px2"]))**2 +
5     (df["py"] - (df["py1"] + df["py2"]))**2 +
6     (df["pz"] - (df["pz1"] + df["pz2"]))**2

```



```

7 )
8
9 # Plot energy and momentum conservation
10 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
11
12 axes[0].hist(df["energy_diff"], bins=100, color="blue", alpha=0.7)
13 axes[0].set_title("Energy Conservation:  $E_{\pi^0} - (E_1 + E_2)$ ")
14 axes[0].set_xlabel("Energy Difference [GeV]")
15 axes[0].set_ylabel("Count")
16
17 axes[1].hist(df["momentum_diff"], bins=100, color="red", alpha=0.7)
18 axes[1].set_title("Momentum Conservation:  $|\vec{p}_{\pi^0} - (\vec{p}_1 + \vec{p}_2)|$ ")
19 axes[1].set_xlabel("Momentum Difference [GeV/c]")
20 axes[1].set_ylabel("Count")
21
22 plt.tight_layout()
23 plt.show()

```

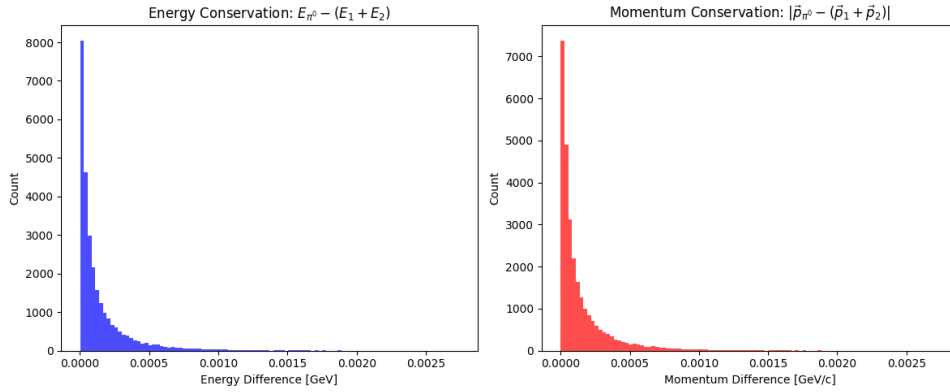


Figure 1: Histograms showing the distribution of energy and momentum differences for the two-body decay.

These graphs can be applied to both the decay of the particles at rest or in motion. The results are the same and we have conservation of energy and momentum (within the range of numerical precision). There is additional validation of almost all the functions and simulation in the code.

## 5 Simulation of the Two-Body Decay, $\pi^0 \rightarrow \gamma\gamma$

The following is a simulation of the two-body decay. It is a vectorized version of the previous code so it can handle a large number of events efficiently. The key steps are:

1. Initialization of variables and arrays
2. For each  $\pi^0$  in `pi0_momenta`, perform a two-body decay to simulate the production of two photons using the `two_body_decay` function. We vectorized this function to handle a large number of events efficiently.
3. Boost to Lab Frame using the `vectorized_lorentz_transform` function

4. Photon Check: an event is detected if at least one of the two photons hit the detector
5. Calculate Fraction

```

1 def vectorized_lorentz_transform(p, v):
2     """
3     Perform a Lorentz boost on a batch of 4-vectors.
4
5     Parameters
6     -----
7     p : numpy.ndarray
8         Array of 4-vectors in the rest frame, shape (N, 4), with
9         columns [E, px, py, pz].
10    v : numpy.ndarray
11        Array of boost velocities, shape (N, 3). (Assuming units where
12        c=1.)
13
14    Returns
15    -----
16    numpy.ndarray
17        Boosted 4-vectors in the lab frame, shape (N, 4).
18    """
19    # Compute beta^2 for each event (v^2)
20    beta2 = np.sum(v**2, axis=1, keepdims=True) # shape: (N,1)
21    # Calculate gamma factor for each event
22    gamma = 1.0 / np.sqrt(1.0 - beta2) # shape: (N,1)
23    # Dot product between the spatial momentum and the boost velocity
24    bp = np.sum(p[:, 1:] * v, axis=1, keepdims=True) # shape: (N,1)
25
26    # Calculate the factor (gamma - 1)/beta2 safely. Set to 0 when
27    # beta2 is nearly zero.
28    with np.errstate(divide='ignore', invalid='ignore'):
29        gamma2 = np.where(beta2 > 1e-12, (gamma - 1.0) / beta2, 0.0) #
30        shape: (N,1)
31
32    # Boost the spatial components:
33    # p[:, 1:] is (N,3) and v is (N,3); the operations are elementwise.
34    spatial = p[:, 1:] + gamma2 * bp * v + gamma * p[:, :1] * v #
35    shape: (N,3)
36    # Boost the energy component:
37    energy = gamma * (p[:, :1] + bp) # shape: (N,1)
38
39    # Concatenate boosted energy and spatial parts along axis 1.
40    boosted = np.hstack([energy, spatial])
41    return boosted
42
43 def estimate_detection_fraction_vectorized(pi0_momenta, M,
44 detector_size=1.0, distance=10.0):
45     """
46     Estimate the fraction of \pi^0 decays for which at least one photon
47     hits the detector.
48
49     The detector is assumed to be a square centered at (x, y) = (0, 0)
50     in a plane
51     located at z = distance. The size of the detector is detector_size
52     (length of a side).

```

```

45 Parameters
46 -----
47 pi0_momenta : numpy.ndarray
48     Lab frame 4-momenta of the \pi^0 particles (shape: [N,4] with
49     columns [E, px, py, pz]).
50 M : float
51     Mass of the \pi^0 particle (in GeV/c^2).
52 detector_size : float, optional
53     Size of the detector (side length), by default 1.0.
54 distance : float, optional
55     z-distance to the detector plane, by default 10.0.
56
57 Returns
58 -----
59 float
60     The estimated geometric efficiency (fraction of decays detected
61     ).
62
63 """
64 n_decays = len(pi0_momenta)
65
66 # Generate random angles for isotropic decays in the pion rest
67 # frame.
68 cos_theta = 2 * np.random.rand(n_decays) - 1
69 theta = np.arccos(cos_theta)
70 phi = 2 * np.pi * np.random.rand(n_decays)
71
72 # In the rest frame of the \pi^0, energy = |p| for a photon. Each
73 # photon gets half the mass.
74 p_mag = M / 2.0
75
76 # Photon 1 in the rest frame: 4-vector [E, px, py, pz]
77 p1_rest = np.column_stack([
78     np.full(n_decays, p_mag),          # Energy
79     p_mag * np.sin(theta) * np.cos(phi),
80     p_mag * np.sin(theta) * np.sin(phi),
81     p_mag * cos_theta
82 ])
83
84 # Photon 2 has exactly the opposite momentum.
85 p2_rest = np.column_stack([
86     np.full(n_decays, p_mag),          # Energy
87     -p_mag * np.sin(theta) * np.cos(phi),
88     -p_mag * np.sin(theta) * np.sin(phi),
89     -p_mag * cos_theta
90 ])
91
92 # Compute the boost velocity of the \pi^0: v = p/E
93 # pi0_momenta has columns [E, px, py, pz]
94 v = pi0_momenta[:, 1:] / pi0_momenta[:, :1] # shape: (N,3)
95
96 # Boost both photons to the lab frame.
97 p1_lab = vectorized_lorentz_transform(p1_rest, v)
98 p2_lab = vectorized_lorentz_transform(p2_rest, v)
99
100 def check_hits(p):
101     """
102     Check if the boosted photon (p) hits the detector.

```

```

99     p is expected to be a numpy array of shape (N,4) for lab-frame
      4-vectors.
100     """
101     # We are interested in the photon's z-component; if it is
      nearly zero or negative (moving backward), it won't hit.
102     pz = p[:, 3]
103     nonzero = np.abs(pz) > 1e-8
104     # For photons moving toward the detector (i.e. pz > 0), compute
      the parametric distance t such that z = distance.
105     # For non-moving or backward-moving photons, set t = infinity.
106     t = np.where((pz > 0) & nonzero, distance / pz, np.inf)
107     # The photon hits at positions (x_hit, y_hit)
108     x_hit = p[:, 1] * t
109     y_hit = p[:, 2] * t
110     # Check if (x_hit, y_hit) fall within half the detector size.
111     hit = (np.abs(x_hit) <= detector_size / 2.0) & (np.abs(y_hit)
      <= detector_size / 2.0)
112     # Only count if the photon is moving forward.
113     hit = hit & (pz > 0)
114     return hit
115
116     hits1 = check_hits(p1_lab)
117     hits2 = check_hits(p2_lab)
118
119     # An event is detected if at least one of the two photons hits the
      detector.
120     detected = hits1 | hits2
121     fraction_detected = np.mean(detected)
122     return fraction_detected

```

Here we apply the functions defined above for our 120 GeV momenta array.

```

1 if __name__ == '__main__':
2     # Define the  $\pi^0$  mass (in GeV/c2)
3     M_pi0 = 0.1349768
4     n_events = 100000 # Number of decays to simulate
5
6     # Generate random lab frame  $\pi^0$  momenta.
7     # We'll sample the spatial momentum components from a normal
      distribution.
8     lab_momenta_spatial = np.random.normal(0, 1, (n_events, 3))
9     # Compute the energy using  $E = \sqrt{p^2 + m^2}$ 
10    energies = np.sqrt(np.sum(lab_momenta_spatial**2, axis=1) + M_pi0
      **2)
11    # Construct the  $\pi^0$  momenta array with columns [E, px, py, pz]
12    pi0_momenta = np.column_stack([energies, lab_momenta_spatial])
13
14    # Estimate the geometric efficiency:
15    fraction = estimate_detection_fraction_vectorized(momenta_array,
      M_pi0,
16
17
18    detector_size
19    =1.0,
20    distance=10.0)
21
22    print(f"Geometric efficiency: {fraction:.6f}")
23    print(f"Expected number of detected events: {fraction * n_events:.0
      f}")

```

We get outputs of around 0.36 or 36%.

We also apply the functions for a isotropic distribution of momenta and get a similar result of 0.16%

```

1 if __name__ == '__main__':
2     # Define the \pi^0 mass (in GeV/c^2)
3     M_pi0 = 0.1349768
4     n_events = 100000 # Number of decays to simulate
5
6     # Generate random lab frame \pi^0 momenta.
7     # We'll sample the spatial momentum components from a normal
8     # distribution.
9     lab_momenta_spatial = np.random.normal(0, 1, (n_events, 3))
10    # Compute the energy using E = sqrt(p^2 + m^2)
11    energies = np.sqrt(np.sum(lab_momenta_spatial**2, axis=1) + M_pi0
12    **2)
13    # Construct the \pi^0 momenta array with columns [E, px, py, pz]
14    pi0_momenta = np.column_stack([energies, lab_momenta_spatial])
15
16    # Estimate the geometric efficiency:
17    fraction = estimate_detection_fraction_vectorized(pi0_momenta,
18    M_pi0,
19    detector_size=1.0,
20    distance=10.0)
21
22    print(f"Geometric efficiency: {fraction:.6f}")
23    print(f"Expected number of detected events: {fraction * n_events:.0f}")

```

## 5.1 Changing the detector coverage

The probability a photon hits the detector is determined by:

$$P_{\text{hit}} \approx \frac{\text{Detector Area}}{4\pi \times (\text{Distance})^2}$$

Our original setup had a detector of  $1\text{ m} \times 1\text{ m}$  at  $10\text{ m}$  which had a solid angle  $\Delta\Omega = \frac{1}{10^2} = 0.01\text{ sr}$  the total solid angle is then  $4\pi \approx 12.566\text{ sr}$  This gives us a probability per photon of :

$$\frac{0.01}{12.566} \approx 0.000796 (0.08\%)$$

▷ Probability for 2 photons:

$$1 - (1 - 0.000796)^2 \approx 0.00159 (0.16\%)$$

### Effect of Detector Size

1. Smaller Detector (e.g.,  $0.1\text{ m} \times 0.1\text{ m}$ ):

▷ Area:  $0.01\text{ m}^2 \rightarrow \Delta\Omega = \frac{0.01}{100} = 0.0001\text{ sr}$

▷ Probability per photon:  $\frac{0.0001}{12.566} \approx 8 \times 10^{-6}$

▷ Expected fraction:  $\approx 1.6 \times 10^{-5} (0.0016\%)$

2. Larger Detector (e.g.,  $10\text{ m} \times 10\text{ m}$ ):

▷ Area:  $100\text{ m}^2 \rightarrow \Delta\Omega = \frac{100}{100} = 1\text{ sr}$

▷ Probability per photon:  $\frac{1}{12.566} \approx 0.0796$

▷ Expected fraction:  $\approx 1 - (1 - 0.0796)^2 \approx 0.153 (15.3\%)$

## 5.2 Simulation Results

To estimate the geometric acceptance, also known as geometric efficiency, of a hypothetical detector for detecting photons resulting from the decay of neutral pions ( $\pi^0 \rightarrow \gamma\gamma$ ). The detector is positioned 10 meters downstream from the target, featuring a 1m x 1m front face.

### Simulation Setup

The simulation was conducted using a vectorized process to model the two-body decay of  $\pi^0$  particles into two photons. Initially, the momenta of the  $\pi^0$  were sourced from a dataset simulating proton-target collisions. These momenta were used as inputs to model the decay events in the rest frame of the  $\pi^0$  particles.

### Lorentz Boost to the Lab frame

Photons arising from these decays were subject to relativistic transformations, as each photon pair was translated from the  $\pi^0$  rest frame to the lab frame using Lorentz boosts. A vectorized method was employed for the transformations, allowing efficient computation across the entire dataset.

### Detection Criteria

For each photon, its trajectory was assessed to determine whether it intersected with the detector's plane at  $z = 10m$ . An intersection was counted as a 'hit' if the coordinates at intersection remained within the  $[-0.5m, 0.5m]$  bounds of the detector face in both x and y dimensions.

The fraction of decays wherein at least one photon intersected with the detector plane defines the geometric acceptance. Across the simulations, the geometric efficiency was calculated as follows:

$$\text{Geometric Efficiency} = \frac{\text{Number of Hits}}{\text{Total Number of Decays}}$$

### Results and Interpretation

The simulation resulted with varying output due to random chance. In the table below, are the results of the geometric efficiency for different outputs with random seeds.

| Geometric Efficiency |
|----------------------|
| 0.367154             |
| 0.366336             |
| 0.367865             |

Table 1: Geometric Efficiency for 120 GeV decays provided by the data file

| Geometric Efficiency |
|----------------------|
| 0.001670             |
| 0.001480             |
| 0.001540             |

Table 2: Geometric Efficiency for Isotropic Decays

When using random isotropic decays, we find the geometric efficiency to be approximately 0.0016. This aligns with the theoretical prediction of 0.16%.

However, when using the data file provided, which came from  $\pi^0$  with average momenta of 120 GeV, we find the geometric efficiency to be approximately 0.367. This is significantly higher than the theoretical prediction of 0.16%. This discrepancy is likely due to the fact that the data file provided is not isotropic, and thus the geometric efficiency is higher.

We can see that the data is not isotropic by simply seeing that the average energy of the photons is 6 GeV and has maximum energy of 108 GeV.

```
1 print(E.mean())
2 print(E.max())
```

output

```
1 6.031258107431741
2 108.53296975773868
```

compared to

```
1 1.602811772293395
2 5.149393972687484
```

of the isotropic data

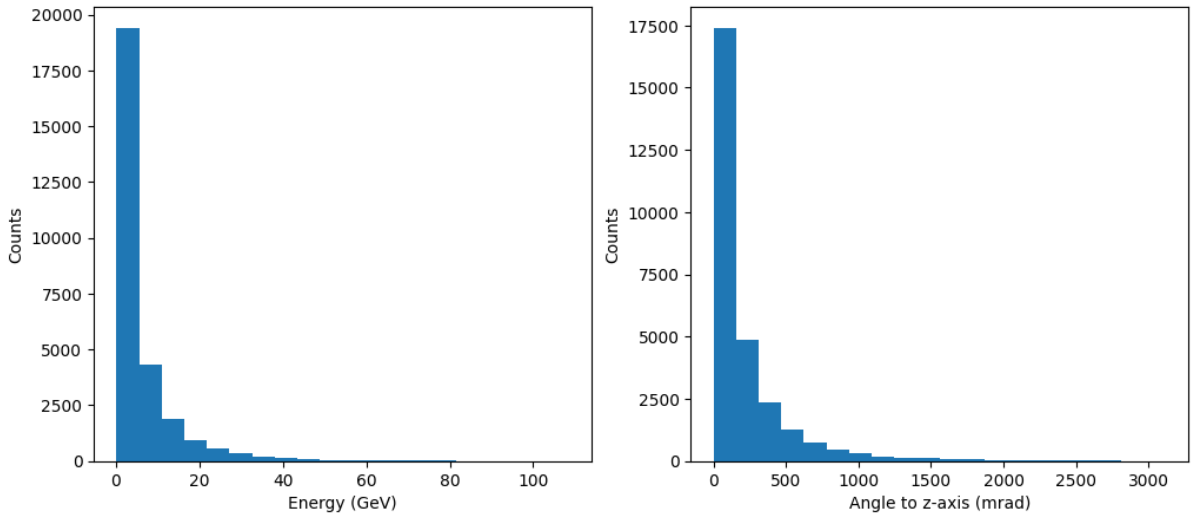


Figure 2: Distribution of the 120 GeV Decay

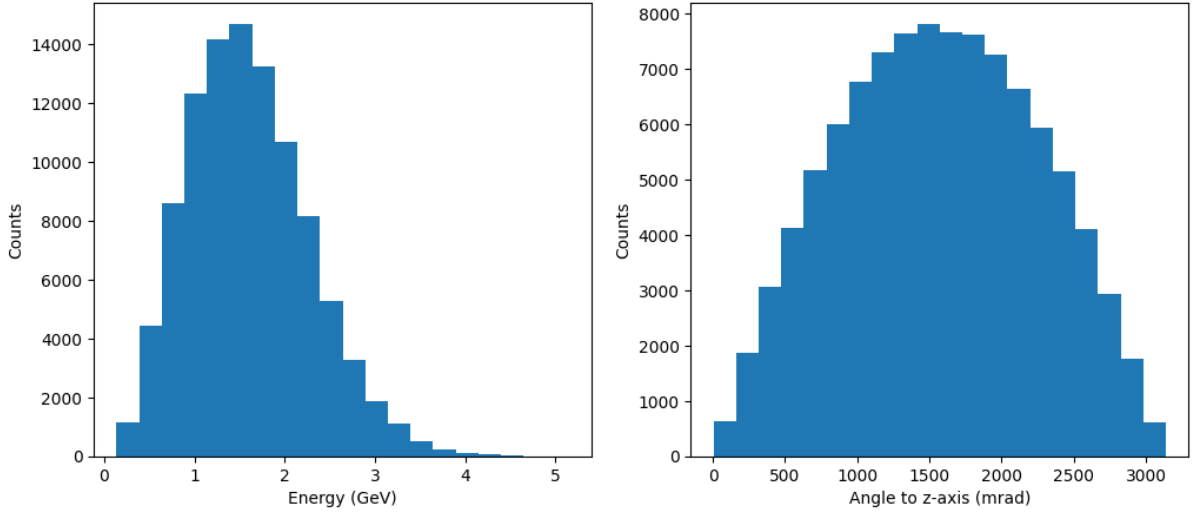


Figure 3: Distribution of the Isotropic Decay

It is clear that the initial momenta and angles of the  $\pi^0$  particles significantly influenced the likelihood of detection, demonstrating the sensitivity of geometric acceptance to the kinematic conditions of particle decays.

### 5.3 Changing Detector Sizes

```

1 results = []
2 configurations = [
3     {"detector_size": 10.0, "distance": 1.0},
4     {"detector_size": 10.0, "distance": 10.0},
5     {"detector_size": 10.0, "distance": 100.0},
6     {"detector_size": 0.1, "distance": 1.0},
7     {"detector_size": 0.1, "distance": 10.0},
8     {"detector_size": 0.1, "distance": 100.0},
9 ]
10
11 for config in configurations:
12     detector_size = config["detector_size"]
13     distance = config["distance"]
14     fraction = estimate_detection_fraction_vectorized(pi0_momenta,
15                                                       M_pi0,
16                                                       detector_size=
17                                                         detector_size
18                                                         ,
19                                                         distance=
20                                                         distance)
21     results.append({
22         "detector_size": f"{detector_size:.1f}m x {detector_size:.1f}m"
23         ,
24         "distance": f"{distance:.1f}m",
25         "efficiency": f"{fraction:.6f}"
26     })
27
28 # Display results in a nice table
29 print("| Detector Size          | Distance | Geometric Efficiency |")

```



```

25 print("|-----|-----|-----|")
26 for result in results:
27     print(f"| {result['detector_size']:<20} | {result['distance']:<8} | {result['efficiency']:<20} |")

```

| Detector Size | Distance (m) | Geometric Efficiency |
|---------------|--------------|----------------------|
| 1.0m x 1.0m   | 10.0         | 0.369180980831       |
| 10.0m x 10.0m | 1.0          | 0.991713787830       |
| 10.0m x 10.0m | 10.0         | 0.912016785803       |
| 10.0m x 10.0m | 100.0        | 0.370852448522       |
| 0.1m x 0.1m   | 1.0          | 0.368398591700       |
| 0.1m x 0.1m   | 10.0         | 0.021551264270       |
| 0.1m x 0.1m   | 100.0        | 0.000284505139       |

Table 3: Geometric Efficiency for Different Detector Configurations

and for the isotropic data:

| Detector Size | Distance (m) | Geometric Efficiency |
|---------------|--------------|----------------------|
| 1.0m x 1.0m   | 10.0         | 0.001630000000       |
| 10.0m x 10.0m | 1.0          | 0.458140000000       |
| 10.0m x 10.0m | 10.0         | 0.086440000000       |
| 10.0m x 10.0m | 100.0        | 0.001460000000       |
| 0.1m x 0.1m   | 1.0          | 0.001510000000       |
| 0.1m x 0.1m   | 10.0         | 0.000010000000       |
| 0.1m x 0.1m   | 100.0        | 0.000000000000       |

Table 4: Geometric Efficiency for Different Detector Configurations

## References

- [1] David J. Griffiths. *Introduction to Elementary Particles*. Wiley-VCH, 2 edition, 2008.
- [2] Particle Data Group. Api. Accessed: 2025-04-09.
- [3] Pasayten Institute. Neutral pion — the field guide to particle physics, 2023. Accessed: 2025-04-09.
- [4] Mark Thomson. *Modern Particle Physics*. Cambridge University Press, 2013.
- [5] Esther Weil, Gernot Eichmann, Christian S. Fischer, and Richard Williams. Electromagnetic decays of the neutral pion. *Physical Review D*, 96(1), July 2017.
- [6] Wikipedia contributors. Pion — wikipedia, the free encyclopedia, 2025. Accessed: 2025-04-09.
- [7] Simon Zimmermann. *Study of neutral pion production in high energy proton-proton collisions at the LHC*. Phd thesis, University of Heidelberg, 2013.